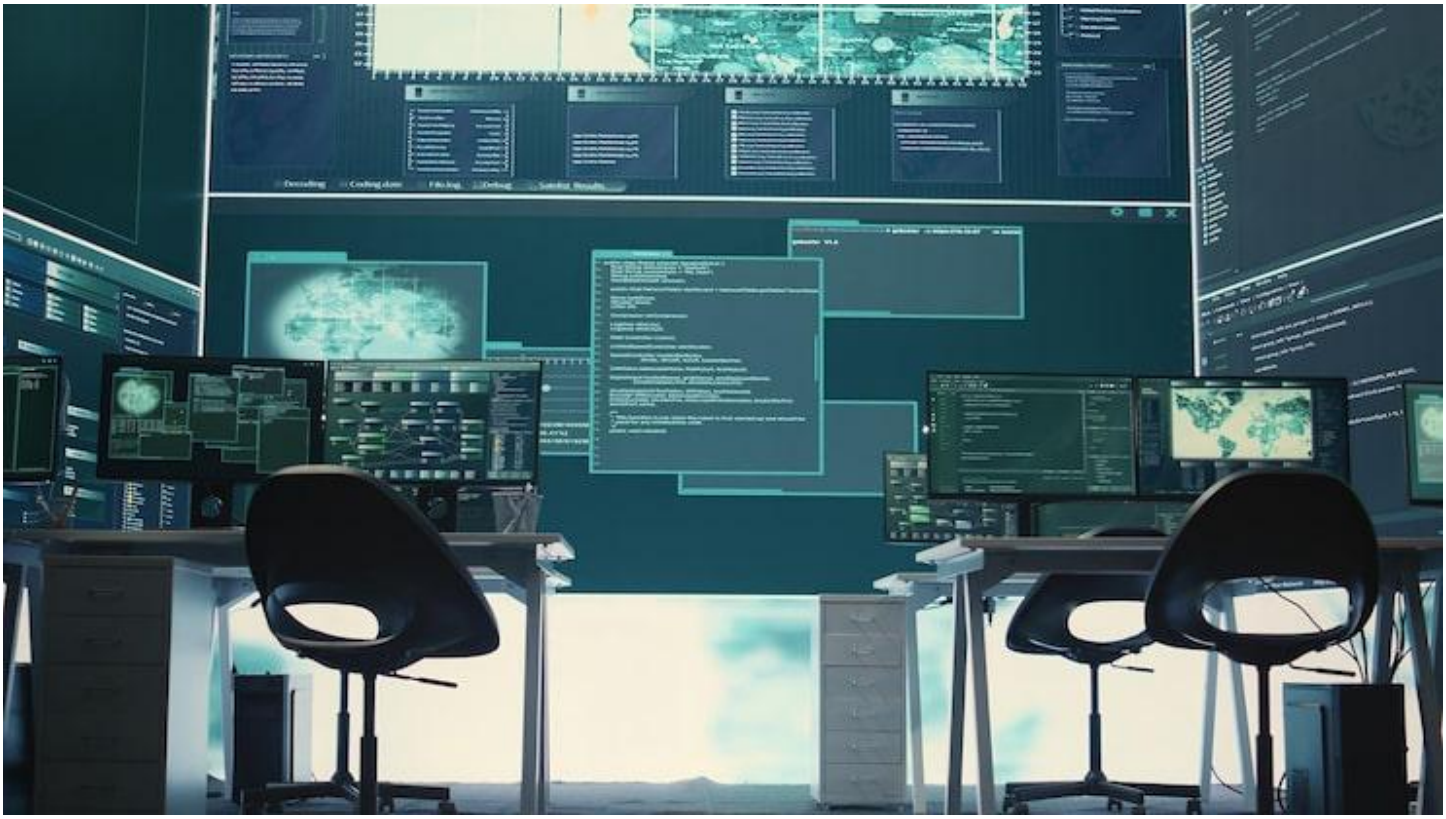


SOC Home Lab

SPLUNK-based Detection

Ramya Radjesh

10/12/26

SOC-Splunk

Project Objective

This Project SOC Home Lab setup built on Splunk Enterprise, designed to practice the daily work flow of SOC Analyst and setup real SOC home lab and to understand the working of Splunk. A three-machine environment — a Splunk indexer, a victim server running a Universal Forwarder, and a Kali Linux attacker — was used to generate genuine attack traffic (SSH brute force, port scanning, privilege escalation, simulated impossible travel) and detect it with custom Splunk Search Processing Language (SPL) queries. The project covers the full detection engineering lifecycle end to end: infrastructure setup, real-time log ingestion, live attack simulation, detection writing, event correlation, real-time alerting, SOC dashboarding, and threat intelligence enrichment.

Contents

Project Objective 1

Phases Summary 3

Learning Objective & Architecture 0

What is Splunk and its uses 2

- Splunk Enterprise 2
- Splunk Universal Forwarder 2

Splunk SPL Commands 3

- SPL Syntax Basics 3
  - Brute Force - Failed Login count per source IP 4
  - Brute Force - Successful Login (correlation) 4
  - Port Scan Detection 5
  - Privilege Escalation 5
  - Impossible Travel (Simulated GeoIP) 5
  - Off-Hours Login (exfiltration proxy) 6
  - Multi-Stage Attack Chain Correlation 7
  - Real-time alerting 8
  - Threat Intel + Asset Enrichment (chained lookups) 10

MITRE ATT&CK Mapping 11

Dashboard 13

Conclusion 14

Phases Summary

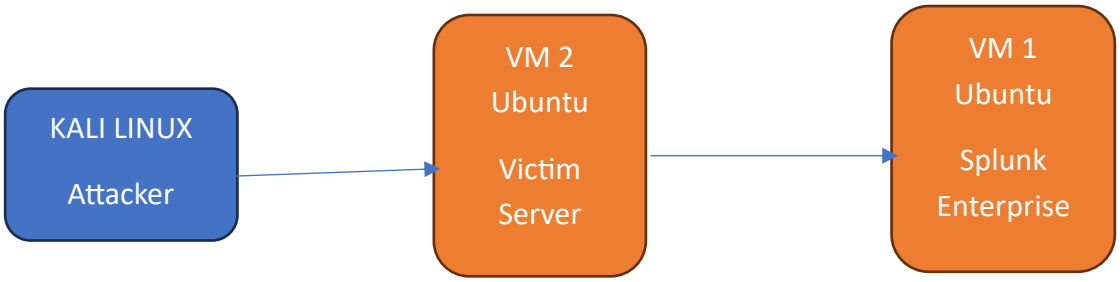
Phase	Description	Status
1. Infrastructure	Deployed Splunk Enterprise, Universal Forwarder, and Kali on 3 VMs; configured networking	Completed
2. Log Ingestion	Real-time forwarding of auth/syslog/UFW logs from VM2 to VM1 over port 9997	Completed
3. Attack Traffic	Real SSH brute force (Hydra) and port scanning (Nmap) run from Kali against VM2	Completed
4. Basic Detections	SPL queries counting failed logins per source IP	Completed
5. Correlation Searches	5 detections: brute force+success, port scan, privilege escalation, impossible travel, multi-stage chain	Completed
6. Real-Time Alerting	Scheduled, throttled alert on brute-force-then-success pattern	Completed
7. Dashboards	5-panel SOC Overview Dashboard in Dashboard Studio, including a MITRE ATT&CK table	Completed
8. Enrichment	Threat intel and asset-context lookup tables joined into detections	Completed

Learning Objective & Architecture

The goal of this project is to understand the daily basis of work life in SOC analyst through an enterprise-level software - Splunk. Here I focused detection engineering skills by running a live, attacker-vs-defender lab rather than working from static sample data.

Lab Design

MACHINE	Role
VM1 (ubuntu)	Splunk Enterprise - Indexer and Search Head
VM2 (ubuntu)	Victim / enterprise server, running a Splunk Universal Forwarder
Kali Linux	Attacker machine



Architecture of this Project

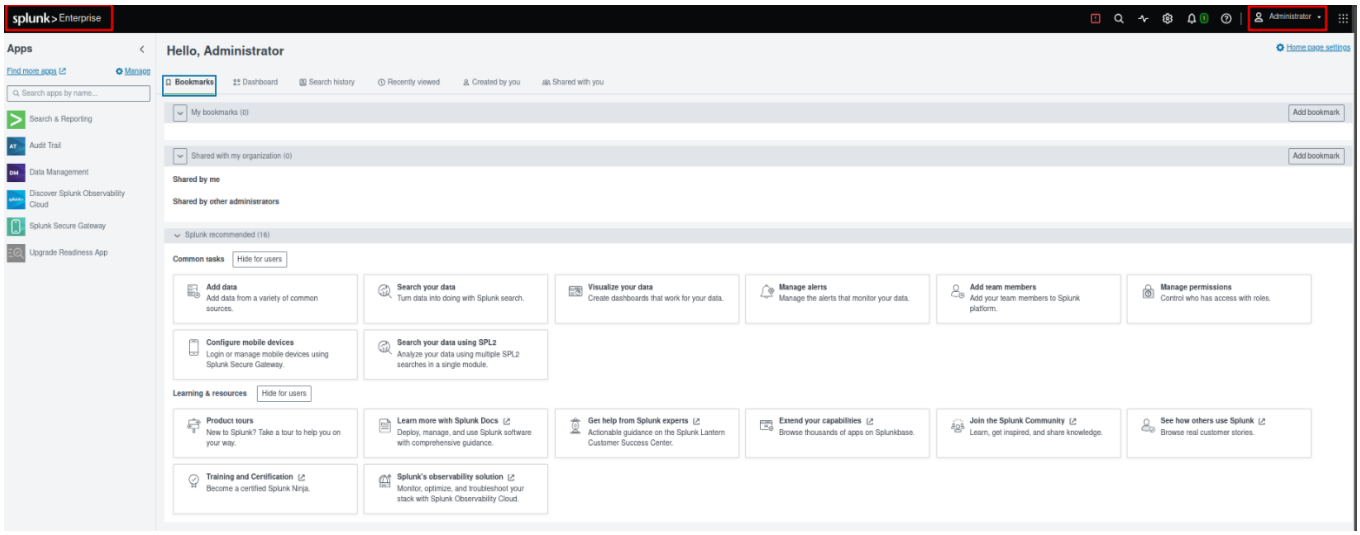
Kali Linux generates real SSH brute-force (Hydra) and Nmap port-scan traffic against VM2. VM2's Universal Forwarder ships `/var/log/auth.log`, `/var/log/syslog`, and `/var/log/ufw.log` in real time over TCP port 9997 to VM1, where Splunk Enterprise indexes and searches the data.

Ports in use: 22 (SSH), 80 (HTTP), 8000 (Splunk Web), 8089 (Splunk management), 9997 (forwarder-to-indexer data).

Lab Infrastructure Evidence:

Ubuntu VM 1 - Splunk Enterprise Machine

```
ramya@ubuntu1-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:90:e6:e7 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet [redacted] brd 192.168.110.255 scope global dynamic noprefixrou
te ens33
       valid_lft 1007sec preferred_lft 1007sec
    inet6 fe80::e788:fe0c:3fc4:ebf9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
ramya@ubuntu1-server:~$
```



Ubuntu VM 2 - Splunk Forwarder Machine

```
ramyasoc@ramyasoc-virtual-machine:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:ef:8b:17 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet [redacted] brd 192.168.110.255 scope global dynamic ens33
       valid_lft 1793sec preferred_lft 1793sec
    inet6 fe80::20c:29ff:feef:8b17/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOW
N group default
    link/ether 42:07:8b:65:e5:e9 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
ramyasoc@ramyasoc-virtual-machine:~$
```

What is Splunk and its uses

Splunk is a platform for collecting, indexing, and searching machine-generated data (logs, events, metrics) at scale, then turning it into real-time dashboards, alerts, and correlation searches via its own query language, SPL (Search Processing Language).

Common enterprise uses:

- **Security (SIEM):** detecting intrusions, correlating attacker behavior, and driving SOC workflows — the focus of this project
- **IT Operations:** monitoring infrastructure health, application performance, and root-causing outages from log data
- **Compliance & Audit:** retaining and searching logs to meet regulatory requirements
- **Business Analytics:** turning operational data (web traffic, transactions, user activity) into business dashboards

This project uses Splunk specifically in its security/SIEM capacity, covering its five core security use cases: threat hunting, anomaly detection, risk-based alerting, cybersecurity framework alignment (MITRE ATT&CK), and threat intelligence enrichment (see the dedicated section below).

Splunk Enterprise

Splunk Enterprise is the core platform: it indexes incoming data, runs SPL searches against it, and serves the Splunk Web interface (dashboards, alerts, lookups, user management). In this project, Splunk Enterprise runs on VM1 and acts as the indexer and search head — the single place all logs are forwarded to, stored, and queried.

Splunk Universal Forwarder

The Universal Forwarder is a lightweight agent installed on a separate machine to collect local log files and ship them to a Splunk Enterprise indexer in real time, without doing any indexing or searching itself. In this project, it runs on VM2 (the victim server), watching `/var/log/auth.log`, `/var/log/syslog`, and `/var/log/ufw.log`, and forwarding them to VM1 over TCP port 9997. This separation — forwarders on endpoints, one central indexer — is the standard pattern for centralized log collection in real enterprise Splunk deployments.

Splunk SPL Commands

SPL Syntax Basics

Elements	Purpose	Examples
index= / source= / host=	Scopes the search to a data source	index=* host="vm2"
"text" or field=value	Filters events by raw text or a field match	"Failed password"
| (pipe)	Passes results from one command into the next	... | stats count
rex field=_raw "regex"	Extracts a new field from raw text using a named capture group	(?<src_ip>\d+\.\d+\.\d+\.\d+)
stats <fn> by <field>	Aggregates events into summary rows	stats count by src_ip
eval <field>=<expr>	Creates or calculates a new field	eval severity="Critical"
where <condition>	Filters the results of a prior command	where count > 3
sort -<field> / head N	Orders results and limits row count	sort -count | head 10
table <fields>	Displays only the named fields, in order	table _time, user, src_ip
lookup <csv> <field> AS <field> OUTPUT <fields>	Joins external reference data onto results	lookup threat_intel.csv ip AS src_ip OUTPUT threat_status

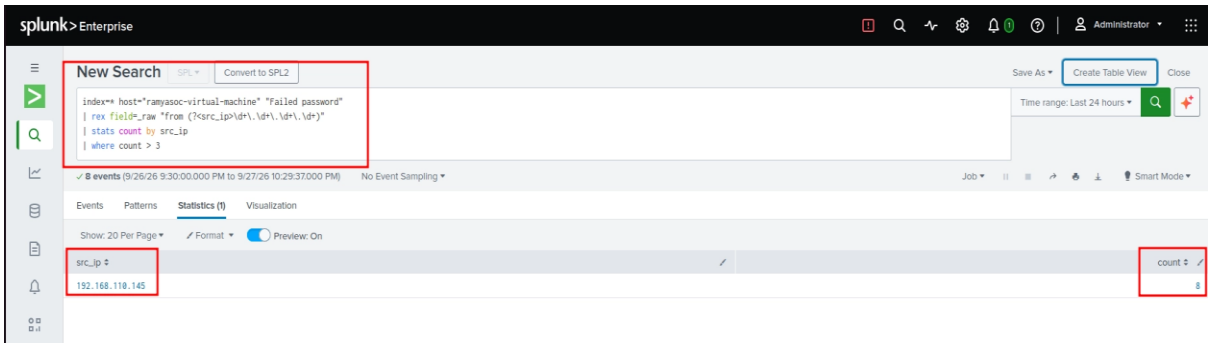
Minimum viable query: a data-source filter, optionally one pipe into a transforming command (stats, table, etc.). Everything else layers on top by chaining more pipes.

SPL Queries Reference

The exact Search Processing Language used for each detection, as run in Splunk's Search & Reporting app:

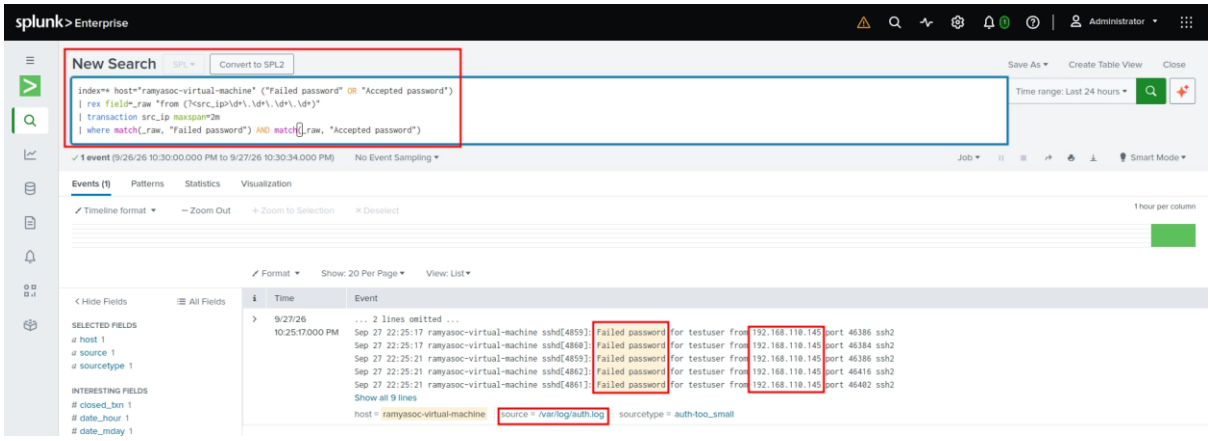
Brute Force - Failed Login count per source IP

```
index=* host="ramyasoc-virtual-machine" "Failed password"
| rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count by src_ip
| where count > 3
```



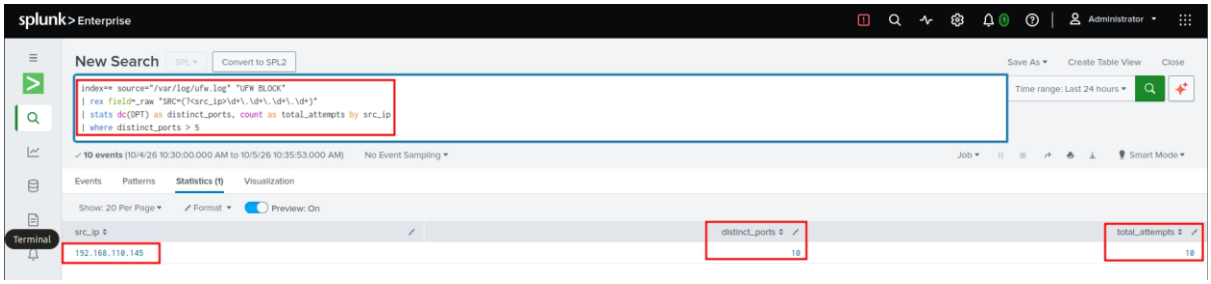
Brute Force - Successful Login (correlation)

```
index=* host="ramyasoc-virtual-machine" ("Failed password" OR "Accepted password")
| rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| transaction src_ip maxspan=2m
| where match(_raw, "Failed password") AND match(_raw, "Accepted password")
```



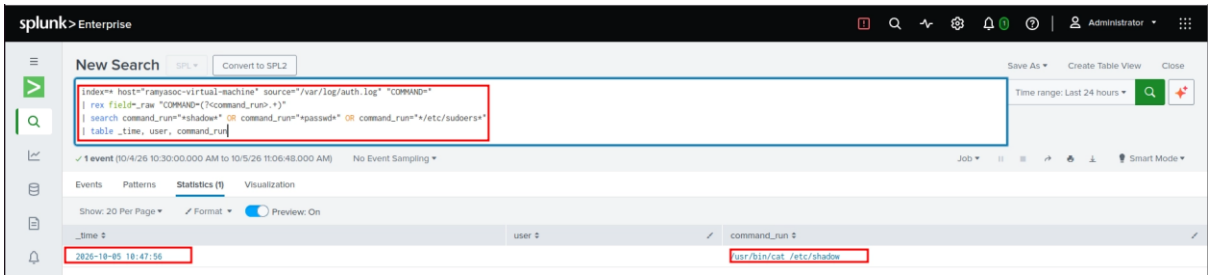
Port Scan Detection

```
index=* source="/var/log/ufw.log" "UFW BLOCK"
| rex field=_raw "SRC=(?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats dc(DPT) as distinct_ports, count as total_attempts by src_ip
| where distinct_ports > 5
```



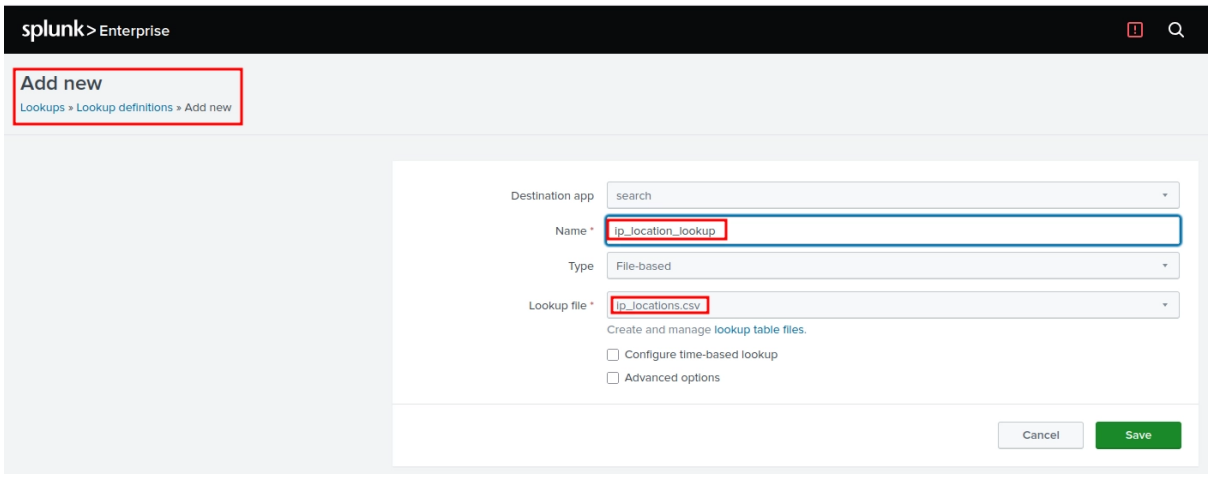
Privilege Escalation

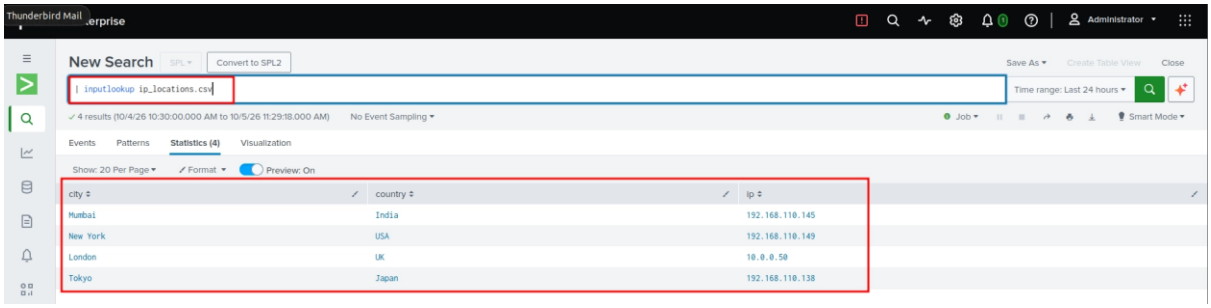
```
index=* host="ramyasoc-virtual-machine" source="/var/log/auth.log" "COMMAND="
| rex field=_raw "sudo:\s+(?<user>\S+)\s+:.+COMMAND=(?<command_run>.+)"
| search command_run="*shadow*" OR command_run="*passwd*" OR command_run="*/etc/sudoers*"
| table _time, user, command_run
```

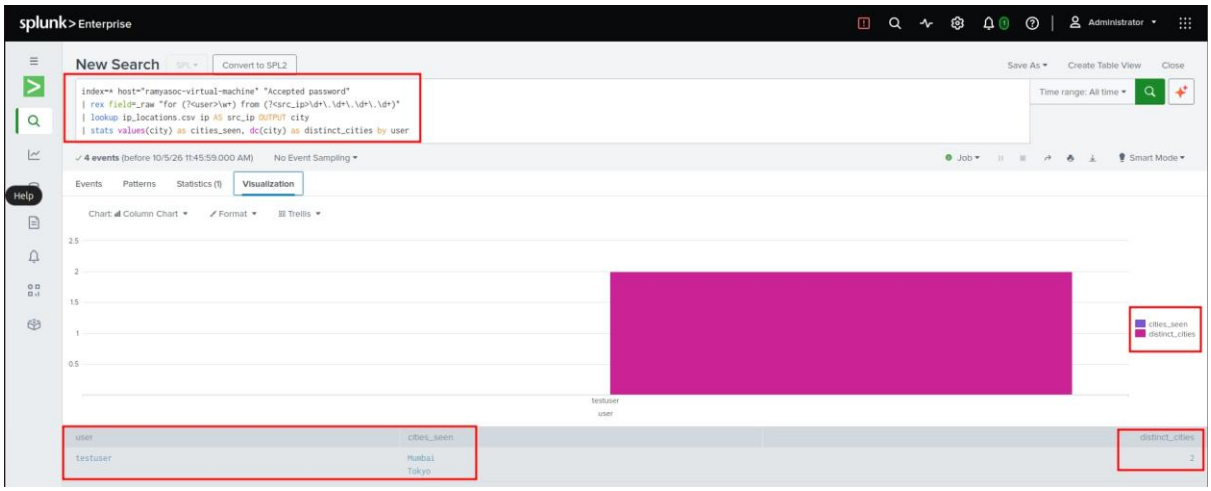


Impossible Travel (Simulated GeoIP)

```
index=* host="ramyasoc-virtual-machine" "Accepted password"
| rex field=_raw "for (?<user>\w+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| lookup ip_locations.csv ip AS src_ip OUTPUT city
| stats values(city) as cities_seen, dc(city) as distinct_cities by user
| where distinct_cities > 1
```

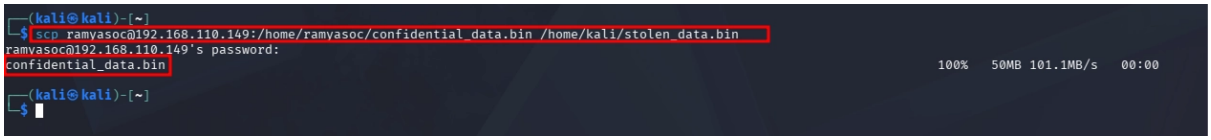


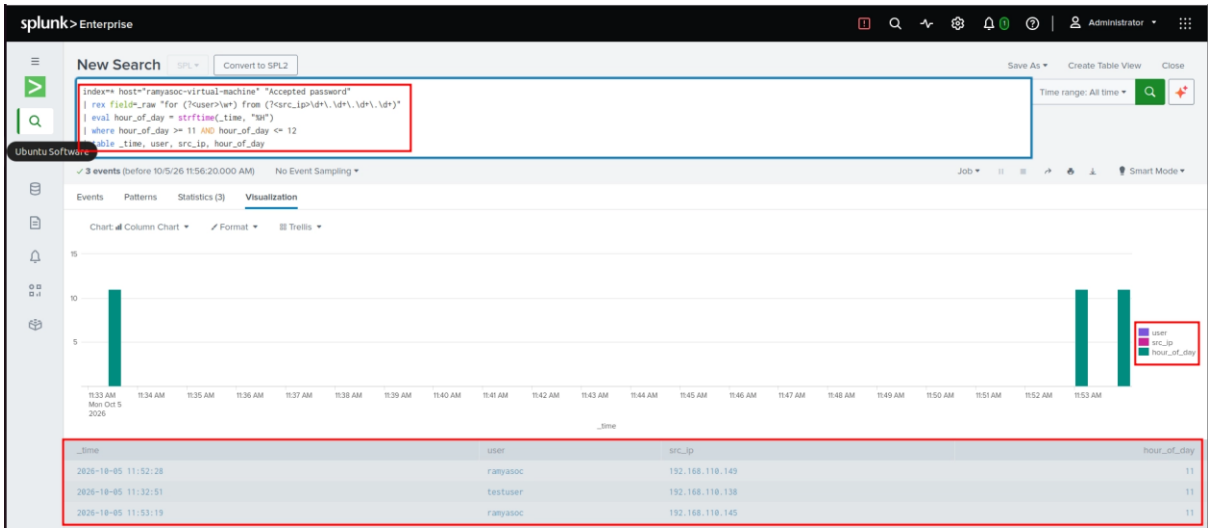




Off-Hours Login (exfiltration proxy)

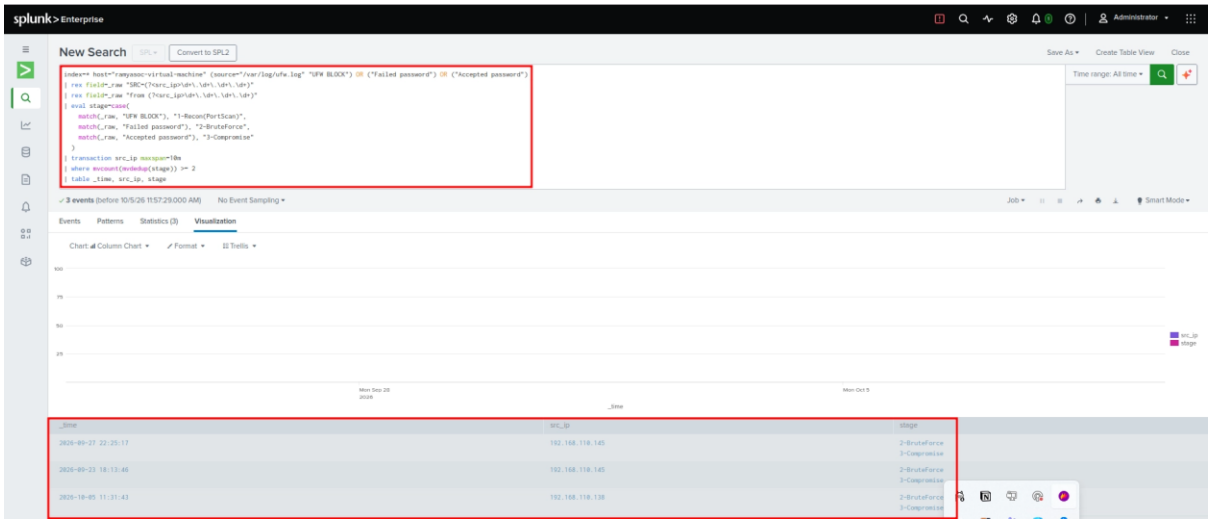
```
index=* host="ramyasoc-virtual-machine" "Accepted password"
| rex field=_raw "for (?<user>\w+) from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| eval hour_of_day = strftime(_time, "%H")
| where hour_of_day < 6 OR hour_of_day > 22
| table _time, user, src_ip, hour_of_day
```





Multi-Stage Attack Chain Correlation

```
index=* host="ramyasoc-virtual-machine" (source="/var/log/ufw.log" "UFW BLOCK") OR ("Failed password") OR ("Accepted password")
| rex field=_raw "SRC=(?<src_ip>\d+\.\d+\.\d+\.\d+)"
| rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| eval stage=case(
   match(_raw, "UFW BLOCK"), "1-Recon(PortScan)",
   match(_raw, "Failed password"), "2-BruteForce",
   match(_raw, "Accepted password"), "3-Compromise"
 )
| transaction src_ip maxspan=10m
| where mvcount(mvdedup(stage)) >= 2
| table _time, src_ip, stage
```

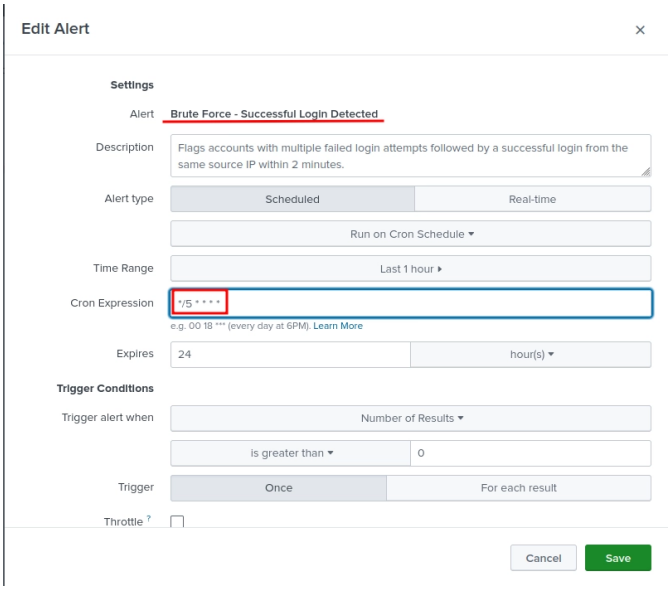


Real-time alerting

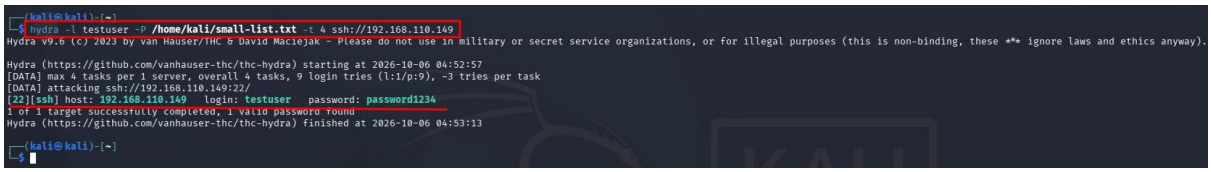
The brute-force-then-success detection was converted from a manual search into a live, scheduled Splunk alert:

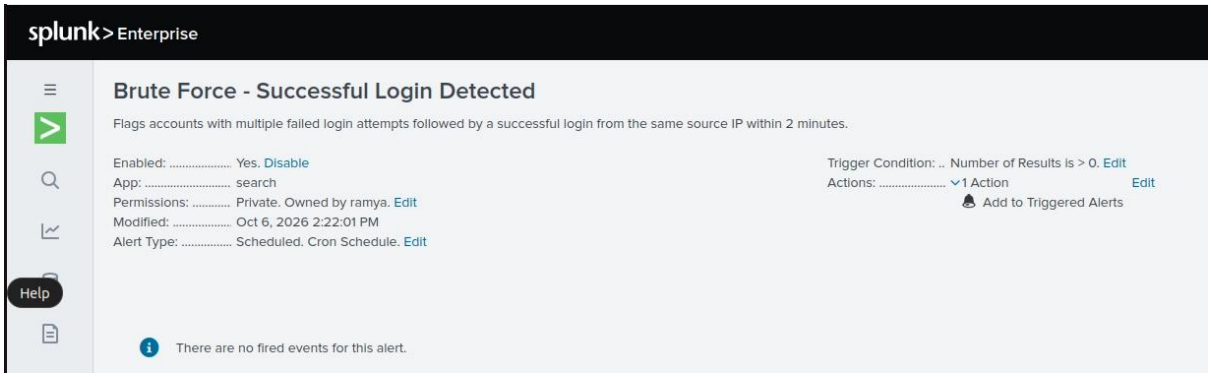
- **Schedule:** cron */5 * * * * (runs every 5 minutes)
- **Trigger condition:** number of results > 0, fired once per run
- **Throttling:** suppressed for 10 minutes after a trigger, to avoid repeated alerts on one ongoing attack
- **Trigger action:** logged to Splunk's Triggered Alerts activity feed

The alert was validated against live hydra attack traffic and confirmed firing automatically on its schedule, with job history showing successful runs roughly every 5 minutes and correct detection counts each time.

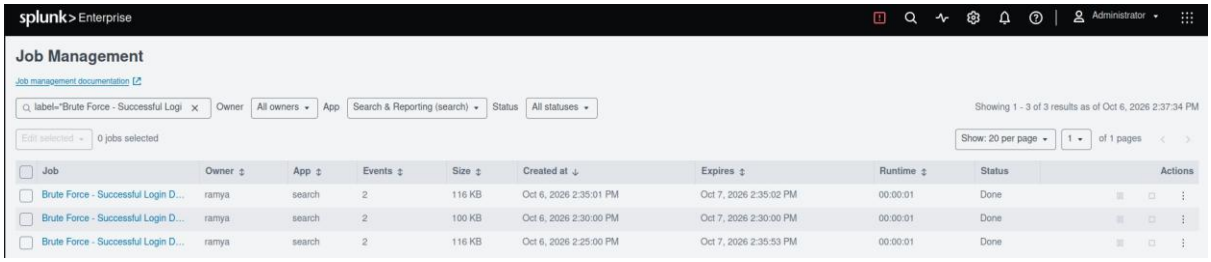


The saved alert's final configuration: cron schedule */5 * * * *, trigger on results > 0, fired once per run.

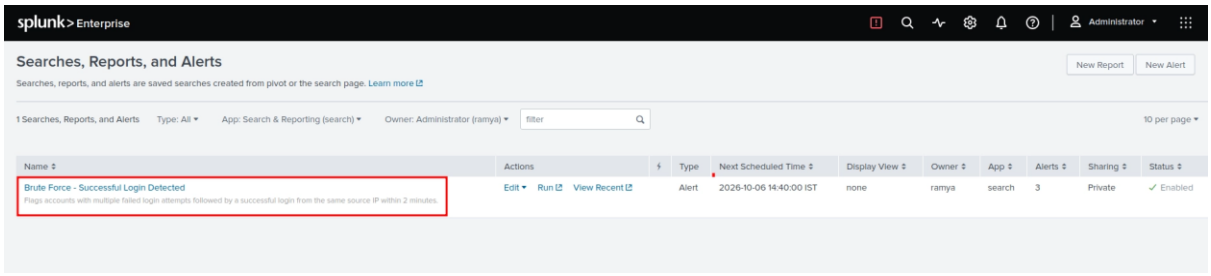




A fresh attack run against VM2 to generate new data for the alert to catch on its next scheduled pass.



The alert's search executing automatically every 5 minutes — 3 completed jobs, each finding 2 matching events, with no manual intervention.



The alert listed as Enabled, with 3 recorded firings and its next scheduled run time.

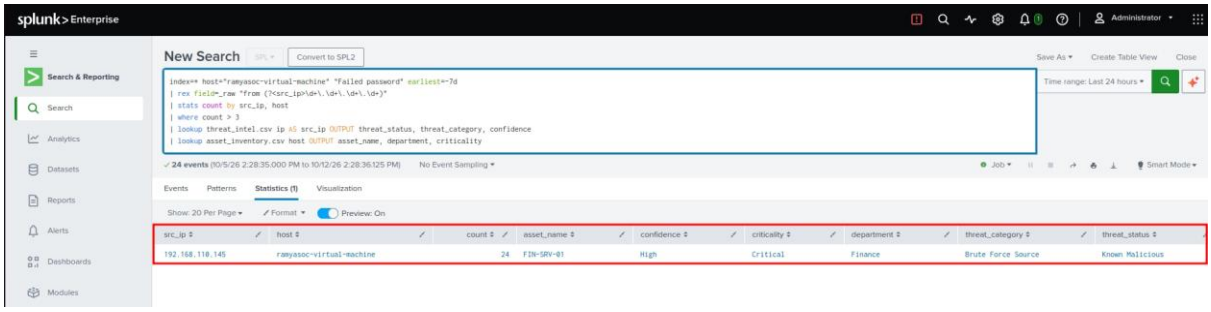
Threat Intel + Asset Enrichment (chained lookups)

Three Splunk lookup tables were built to enrich raw detections with context, simulating real SOC threat-intel and CMDB integration:

Lookup	Purpose
ip_locations.csv	Simulated GeoIP data, used for impossible-travel detection
Threat_intel.csv	Attacker IP reputation (threat status, category, confidence)
asset_inventory.csv	Business asset context (asset name, department, criticality) per host

Chaining two lookups onto the brute-force detection produces a single enriched row showing both attacker reputation and target business impact, e.g.: source IP 192.168.110.145 → **Known Malicious / Brute Force Source / High confidence** attacking host ramyasoc-virtual-machine → **FIN-SRV-01 / Finance / Critical asset**. This mirrors how a real SOC triage view prioritizes alerts by combining who is attacking with what they are attacking.

```
index=* host="ramyasoc-virtual-machine" "Failed password" earliest=-7d
| rex field=_raw "from (?<src_ip>\d+\.\d+\.\d+\.\d+)"
| stats count by src_ip, host
| where count > 3
| lookup threat_intel.csv ip AS src_ip OUTPUT threat_status, threat_category, confidence
| lookup asset_inventory.csv host OUTPUT asset_name, department, criticality
```

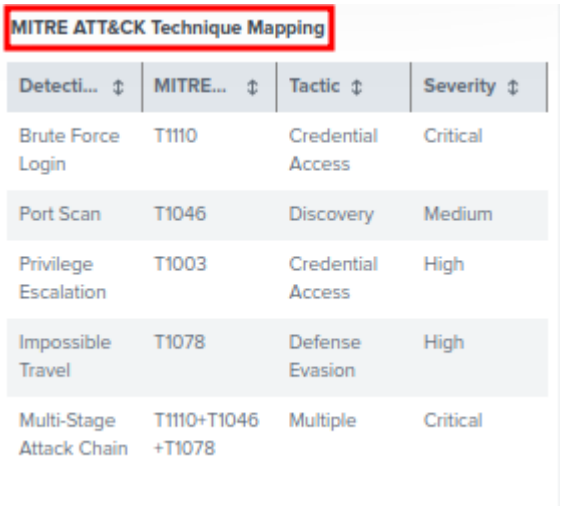


MITRE ATT&CK Mapping

MITRE ATT&CK is a publicly maintained knowledge base of real-world attacker tactics and techniques (e.g. T1110 = Brute Force), organized by the tactic they serve (Credential Access, Discovery, Defense Evasion, etc.). Mapping detections to it is a SOC standard practice — it lets analysts describe an alert in a shared, industry-wide vocabulary instead of a one-off description, and makes it easy to see which parts of the attack lifecycle are and aren't covered by current detections.

```
| makeresults
| eval data="Brute Force Login,T1110,Credential Access,Critical;Port Scan,T1046,Discovery,Medium;Privilege Escalation,T1003,Credential Access,High;Impossible Travel,T1078,Defense Evasion,High;Multi-Stage Attack Chain,T1110+T1046+T1078,Multiple,Critical"
| makemv delim=";" data
| mvexpand data
| eval Detection=mvindex(split(data, ","), 0),
    MITRE_Technique=mvindex(split(data, ","), 1),
    Tactic=mvindex(split(data, ","), 2),
    Severity=mvindex(split(data, ","), 3)
| table Detection, MITRE_Technique, Tactic, Severity
```

Unlike the other panels, this one has no underlying log data — `makeresults` generates a single blank row to build on, which is the standard way to create a static/reference panel in Splunk. The mapping string is packed into one field, split into a list with `makemv`, unrolled into 5 separate rows with `mvexpand`, then each row's four values are pulled into their own columns with `mvindex(split(...))` before the final `table`.



MITRE ATT&CK Technique Mapping

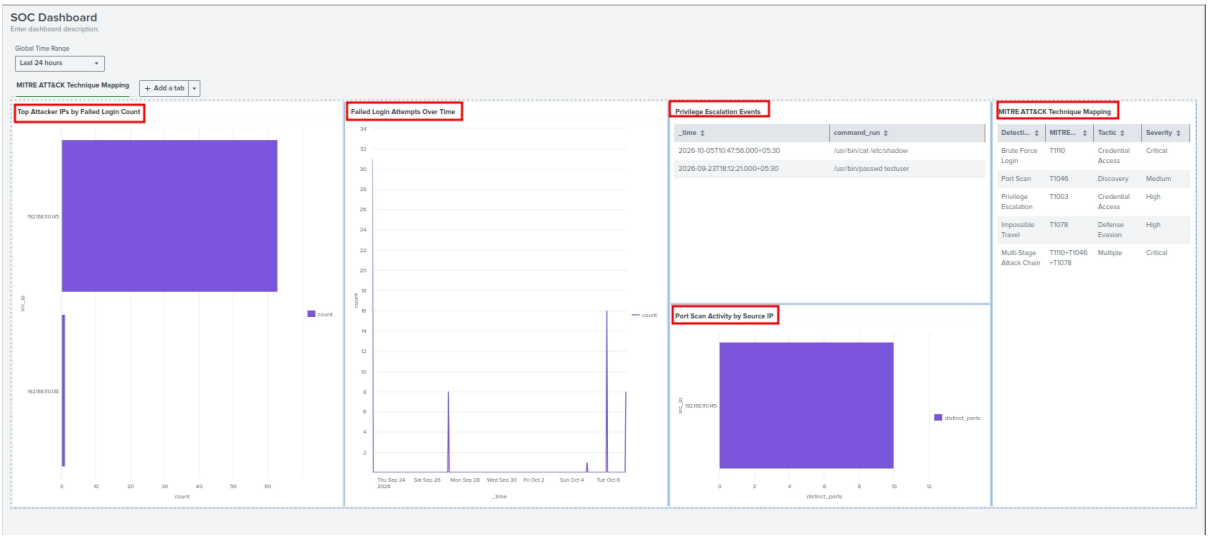
Detecti...	MITRE...	Tactic	Severity
Brute Force Login	T1110	Credential Access	Critical
Port Scan	T1046	Discovery	Medium
Privilege Escalation	T1003	Credential Access	High
Impossible Travel	T1078	Defense Evasion	High
Multi-Stage Attack Chain	T1110+T1046 +T1078	Multiple	Critical

Detection	SPL Logic Summary	MITRE Technique	Tactic	Severity
Brute Force → Successful Login	transaction groups failed/accepted login events per source IP within a 2-minute window; keeps only groups containing both	T1110 — Brute Force	Credential Access	Critical
Port Scan Detection	stats dc(DPT) counts distinct destination ports per source IP from UFW block logs; flags >5 ports	T1046 — Network Service Discovery	Discovery	Medium
Privilege Escalation	Extracts sudo COMMAND text; flags access to /etc/shadow, /etc/passwd, /etc/sudoers	T1003 — OS Credential Dumping	Credential Access	High
Impossible Travel (simulated)	lookup enriches source IP with a simulated city; flags one user logging in from >1 city	T1078 — Valid Accounts	Defense Evasion / Persistence	High
Multi-Stage Attack Chain	eval case() tags each event's stage (recon/brute force/compromise); transaction groups by IP and keeps chains with ≥2 stages	T1110 + T1046 + T1078 (combined)	Multiple	Critical

Dashboard

A 5-panel "SOC Overview Dashboard" was built in Splunk Dashboard Studio:

1. **Top Attacker IPs by Failed Login Count** — bar chart ranking source IPs by failure volume
2. **Failed Login Attempts Over Time** — timechart showing attack activity across the project's testing sessions
3. **Port Scan Activity by Source IP** — bar chart of distinct ports probed per IP
4. **Privilege Escalation Events** — table of sensitive sudo command executions with timestamps
5. **MITRE ATT&CK Technique Mapping** — static reference table (built with makeresults/mvexpand) tying each detection to its technique, tactic, and severity



The finished dashboard, read left to right as a single incident story:

- **Top Attacker IPs by Failed Login Count** — 192.168.110.145 (Kali) is overwhelmingly the dominant source, confirming it as the attacker
- **Failed Login Attempts Over Time** — spikes line up exactly with each hydra run across the project's testing sessions, showing attack activity rather than steady background noise
- **Privilege Escalation Events** — the same IP's activity preceded real post-compromise behavior: reading /etc/shadow and resetting testuser's password
- **Port Scan Activity by Source IP** — the same attacker also ran reconnaissance (10 distinct ports touched), tying scanning to the brute force

- **MITRE ATT&CK Technique Mapping** — ties all of the above back to named techniques (T1110, T1046, T1003, T1078) and tactics, so the dashboard reads as a mapped incident, not just a set of unrelated charts

Together, the five panels tell one coherent story: a single external IP scanned the server, brute-forced a login, got in, and then touched sensitive files — exactly the kind of correlated view a SOC analyst would use to triage an incident at a glance.

Each panel is backed by a working SPL query, so the dashboard reflects live, queryable data rather than static screenshots.

Conclusion

All 9 phases of this project are complete, end to end: a live three-machine SOC lab that generates real attacks, detects them with custom SPL, correlates them into incidents, alerts on them automatically, visualizes them on a dashboard, and enriches them with threat intelligence. Nothing here was simulated with sample data — every detection was built against genuine attacker traffic captured from a real SSH brute force, a real port scan, and real privilege-escalation commands, then validated by watching it land in Splunk and fire a real alert.

The result is a working demonstration of the full detection engineering lifecycle, plus the kind of infrastructure troubleshooting (stale firewall rules, DNS failures, permission errors) that only shows up when you build something yourself rather than follow a script. That combination — detection logic plus the ability to keep the pipeline running when it breaks — is the core of what a SOC analyst does day to day.